Assignment02

Piper Lentz

September 2026

Problem 1: Bubble Sort

Approach

This sorting algorithm, bubble sort, will pass through the array over and over, comparing adjacent indices. If the i+1 index's value is greater than the i index's value, the values swap placement in the array. This algorithm passes through the array, swapping placements of values, until all the values are in order.

Pseudocode

```
function bubbleSort(array, n):
    for i from 0 to n-2:
        swapped = False
        for j from 0 to n-2-i:
            if array[j] > array[j+1]:
                swap(array[j], array[j+1])
                swapped = True
        if swapped = False then
            break
    return
```

Execution

```
(base) piperlentz@Pipers-MacBook-Pro-
Original: 34 7 23 32 5 62 14 19
Pass 1: 7 23 32 5 34 14 19 62
Pass 2: 7 23 5 32 14 19 34 62
Pass 3: 7 5 23 14 19 32 34 62
Pass 4: 5 7 14 19 23 32 34 62
Pass 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Total comparisons: 25
Total swaps: 15
```

Figure 1: Starting with unsorted array.

```
(base) piperlentz@Pipers-MacBook-P
Original: 5 7 14 19 23 32 34 62
Pass 1: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Total comparisons: 7
Total swaps: 0
```

Figure 2: Starting with sorted array.

On an already sorted array, bubble sort makes n-1 comparisons (in this case 7 on a size 8 array), 0 swaps, and 1 pass. It only needs 1 pass because the numbers are already sorted - it does not need to recheck a value's position after has moved because nothing moved in the first place. There are n-1 comparisons because that is the number of adjacent pairs you can make out of an array.

Problem 2: Insertion Sort

Approach

This sorting algorithm, insertion sort, goes through an array starting from the beginning, and checks whether the element is less than the element before it. If so, it swaps their places, and continues doing this until that element is

greater than the one before it. In total, this method only does one pass of the full array.

Pseudocode

```
function insertionSort(array, n):
    for i from 0 to n-1:
        key = array[i]
        j = i-1
        while j >= 0 and array[j] > key:
            array[j+1] = array[j]
            j = j-1
        array[j+1] = key
    retun array
```

Execution

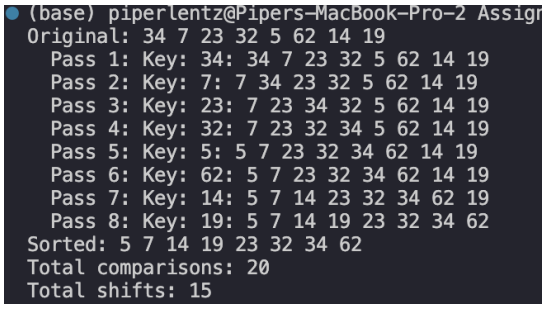


Figure 3: Starting with unsorted array.

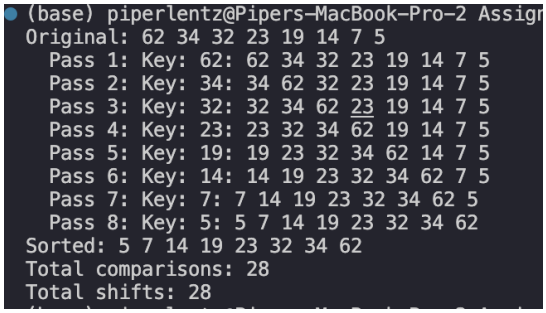


Figure 4: Starting with array in descending order.

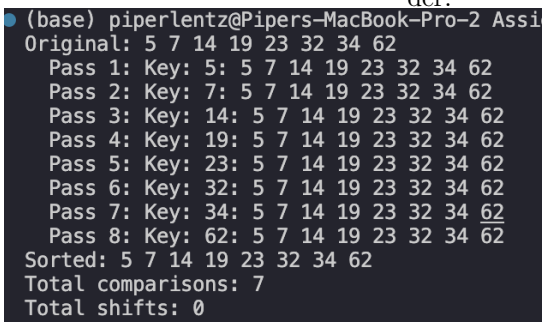


Figure 5: Starting with sorted array.

Which ordering of the same eight numbers makes Insertion Sort do the fewest shifts, and which makes it do the most? Run your program on both orderings, report the counts, and explain the result.

When the numbers are in descending order, insertion sort makes the most shifts (a total of 28 shifts and comparisons for an array of size 8, shown in Figure 4). There is a shift for every comparison. This happens because the key element must be shifted with every element before it.

The fewest shifts in insertion sort takes place when the values are already in order (a total of 0 shifts and 7 comparisons for a size 8 array, shown in Figure 5). n-1 comparisons are made, but in each case the key is greater than the element before it so zero shifts take place.

Problem 3: Selection Sort

Approach

This sorting algorithm, selection sort, starts by calling the first index the minimum index. Then, it goes through the rest of the array's elements and finds the minimum value and its index. If that minimum is less than the value of at the minimum index (the index 0 on the first pass), they swap places. Next, the minimum index moves up an element (so index 1 on round 2, index 2 on round 3...) and the process repeats.

Pseudocode

```
function selectionSort(arr, n):
    for i from 0 to n-2:
        minIdx = i
        for j from i+1 to n-1:
            if arr[j] < arr[minIdx]:
                minIdx = j
        if minIdx != i:
            swap(arr[i], arr[minIdx])
    return arr
```

Execution

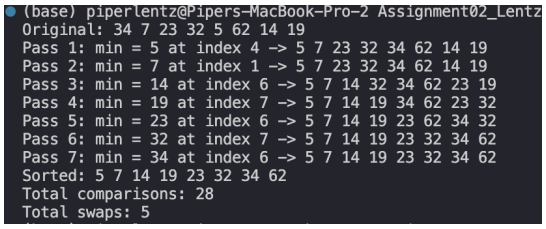


Figure 6: Starting with unsorted array.

Compare the number of swaps with your result for Bubble Sort in Problem 1. Which algorithm makes fewer swaps on A, and why?

For the given array A of size 8, selection sort made 28 comparisons and 5 swaps. On the other hand, bubble sort made 25 comparisons and 15 swaps. Selection sort made fewer swaps because instead of taking every adjacent pair and swapping if element i+1 is smaller than i (like bubble sort), it always swaps the next smallest element into the correct position (it finds the minimum of the remaining part of the array and puts that minimum into the next smallest position).

Problem 4: Quick Sort

Approach

Quick sort chooses a pivot point, typically the last element in an array, and goes through the array determining if elements should be placed to the left or right of the pivot point. Values greater than the pivot will go to the right (later in the array) and values less than the pivot will be to the left (earlier in the array). However, the values may not be in order after one run through (the left piece of the array with pivot value 18 may be for example 7, 5, 8 instead of 5, 7, 8), so this process continues on the smaller pieces of the full array until every value is in the correct position.

Pseudocode

```
function quickSort(arr, low, high):
    if low < high:
```

```
        p = partition
        quickSort(arr, low, p-1)
        quickSort(arr, p+1, high)

function partition(arr, low, high):
    pivot = arr[high]
    i = low-1
    for j from low to high-1:
        if arr[j] <= pivot
            i = i+1
            swap(arr[i], arr[j])
    swap (arr[i+1], arr[high])
    return i+1
```

Execution

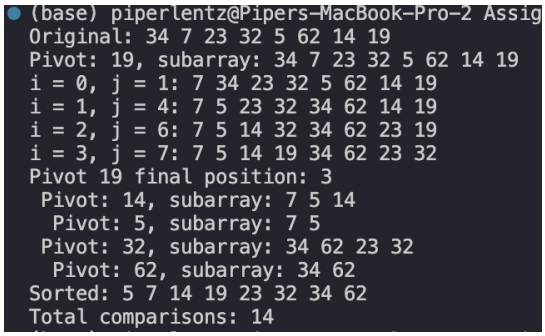


Figure 7: Starting with unsorted array.

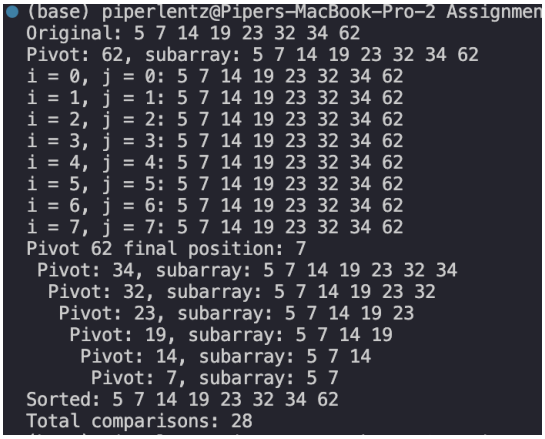


Figure 8: Starting with a sorted array.

Explain why a sorted array is the worst case when the last element is the pivot, and suggest one way to avoid it.

A sorted array is the worst case when the last element is the pivot because everything that is compared to the pivot will be less than it. Therefore, there are no branching sub-arrays, just the one branch with the remaining array elements. The pivot then has to be compared to every remaining array element, resulting in the maximum amount of comparisons (literally every number gets compared to all the others). To fix this, I suggest we can pick a pivot in the middle of the array. This way, if the array is sorted, not every number must be compared to all the others.

Problem 5: Merge Sort

Approach

Merge sort breaks an array in half, then continues breaking each half in half until every array has one element (a single element is already sorted). Then, it merges pairs of sorted pieces of the array together into bigger sorted pieces. This is efficient because while merging two sorted arrays, it only has to compare the first elements in the arrays (we know they are the smallest in value). This merging is repeated until just one sorted array remains.

Pseudocode

```
function mergeSort(arr, low, high):
    if low < high:
        mid = [(low+high)/2]
        mergeSort(arr, low, mid)
```

```
        mergeSort(arr, mid+1, high)
        merge(arr, low, mid, high)

function merge(arr, low, mid, high):
        L = arr[low...mid]
        R = arr[mid+1...high]
        i = 0, j = 0, k= low
        while i < size(L) and j < size(R):
            if L[i] ≤ R[j]:
                arr[k] = L[i]
                i+=1
            else:
                arr[k] = R[j]
                j += 1
            k += 1
        while i < size(L):
            arr[k] = L[i]
            i += 1
            k += 1
        while j < size(R):
            arr[k] = R[j]
            j += 1
            k += 1
    end
```

Execution

```
(base) piperlentz@Pipers-MacBook-Pro-2 Assignment
Original: 34 7 23 32 5 62 14 19
Split: 34 7 23 32 5 62 14 19
Split: 34 7 23 32
Split: 34 7
Merged: 7 34
Split: 23 32
Merged: 23 32
Merged: 7 23 32 34
Split: 5 62 14 19
Split: 5 62
Merged: 5 62
Split: 14 19
Merged: 14 19
Merged: 5 14 19 62
Merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Total comparisons: 17
Comparisons in final merge: 7
```

Figure 9: Starting with unsorted array.

```
(base) piperlentz@Pipers-MacBook-Pro-2 Assig
Original: 5 7 14 19 23 32 34 62
Split: 5 7 14 19 23 32 34 62
Split: 5 7 14 19
Split: 5 7
Merged: 5 7
Split: 14 19
Merged: 14 19
Merged: 5 7 14 19
Split: 23 32 34 62
Split: 23 32
Merged: 23 32
Split: 34 62
Merged: 34 62
Merged: 23 32 34 62
Merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Total comparisons: 12
Comparisons in final merge: 4
```

Figure 10: Starting with a sorted array.

```
(base) piperlentz@Pipers-MacBook-Pro-2 Assignme
Original: 62 34 32 23 19 14 7 5
Split: 62 34 32 23 19 14 7 5
Split: 62 34 32 23
Split: 62 34
Merged: 34 62
Split: 32 23
Merged: 23 32
Merged: 23 32 34 62
Split: 19 14 7 5
Split: 19 14
Merged: 14 19
Split: 7 5
Merged: 5 7
Merged: 5 7 14 19
Merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Total comparisons: 12
Comparisons in final merge: 4
```

Figure 11: Starting with a descending sorted array.

Run your program on a sorted copy and on a reversed copy of A. Explain why the number of comparisons changes much less than for Bubble Sort or Quick Sort, and state how much extra memory Merge Sort needs.

Merge sort's comparison count doesn't vary much with the order of the array because it always splits the array in the same way, so only the work inside each merge changes (ascending order: 12 comparisons, descending order: 12 comparisons). When the input is sorted in ascending order, every element in the left half is smaller than those in the right half, so each merge takes all of the left half first, then copies in the right half with no further comparisons. When the array is in reversed order, the right half is entirely smaller, so the right half empties first and the left half gets copied in with no further comparisons.

Meanwhile, the number of comparisons in bubble sort vary a lot because it sifts through the array over and over comparing adjacent elements until the array is sorted. If the array is already sorted, only n-1 comparisons are needed. If the array is in descending order, every element will end up being compared to all others (28 comparisons when the array has 8 elements).

Quick sort also varies a lot with array order. With a first or last element pivot, a sorted or reversed array results in the worst possible branches - each one splits off a single element and leaves $n - 1$ on the other side. Merge sort avoids this because it always splits at the middle index, so its comparison count stays around the same number.

Merge sort needs O(n) extra memory because each merge copies its subarray into the temporary arrays L and R, so the top-level merge uses n extra ints (in our case 8). This is its disadvantage compared with in-place sorting

methods like bubble sort and quick sort.

Problem 6: Heap Sort

Approach

Heap sort works by storing the array as a binary max-heap (think of it like a binary tree). In this max-heap, every parent (element i) must be greater than or equal to its child (elements 2i+1 and 2i+2). If the parent is not greater than the child, they swap places so that the parent is greater. When the root (the maximum) is successfully at the top of the binary tree (front of the max-heap), it gets swapped to the end of the array. Then another max-heap is made of the remaining array elements, and the process repeats.

Pseudocode

```
function heapSort(arr, n):
    for i from (n/2 -1) down to 0:
        siftDown(arr, i, n)
    for end from n-1 down to 1:
        swap(arr[0], arr[end])
        siftDown(arr, 0, end)

function siftDown(arr, root, size):
    while 2*root +1 < size:
        child = 2*root +1
        if child +1 < size and arr[child+1] > arr[child]:
            child += 1
        if arr[root]≥arr[child]:
            break
        swap(arr[root], arr[child])
        root = child
```

Execution

```
(base) piperlentz@Pipers-MacBook-Pro-2 Assignment
Original: 34 7 23 32 5 62 14 19
Max heap: 62 32 34 19 5 23 14 7
End = 7
Heap: 34 32 23 19 5 7 14
Sorted: 62
===============
End = 6
Heap: 32 19 23 14 5 7
Sorted: 34 62
===============
End = 5
Heap: 23 19 7 14 5
Sorted: 32 34 62
===============
End = 4
Heap: 19 14 7 5
Sorted: 23 32 34 62
===============
End = 3
Heap: 14 5 7
Sorted: 19 23 32 34 62
===============
End = 2
Heap: 7 5
Sorted: 14 19 23 32 34 62
===============
End = 1
Heap: 5
Sorted: 7 14 19 23 32 34 62
===============
Final Sorted: 5 7 14 19 23 32 34 62
Total swaps inside siftDown: 12
```

Figure 12: Starting with unsorted array.

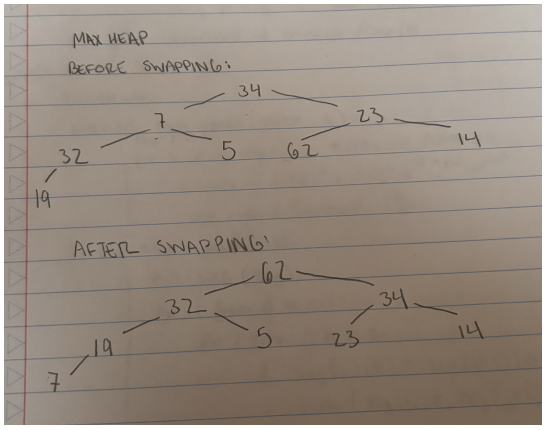


Figure 13: Starting with unsorted array from Figure 12. Heap drawn as a binary tree.

```
(base) piperlentz@Pipers-MacBook-Pro-2 Assignment02_Ler
Original: (34,A) (7,A) (7,B) (32,C) (7,C)
Max heap: (34,A) (32,C) (7,B) (7,A) (7,C)
End = 4
Heap: (32,C) (7,C) (7,B) (7,A)
Sorted: (34,A)
===============
End = 3
Heap: (7,A) (7,C) (7,B)
Sorted: (32,C) (34,A)
===============
End = 2
Heap: (7,B) (7,C)
Sorted: (7,A) (32,C) (34,A)
===============
End = 1
Heap: (7,C)
Sorted: (7,B) (7,A) (32,C) (34,A)
===============
Final Sorted: (7,C) (7,B) (7,A) (32,C) (34,A)
Total swaps inside siftDown: 2
```

Figure 14: Starting with a an array of tuples with duplicates. Shows heap sort is unstable.

Is Heap Sort stable? Explain your answer in your report using a small example array that contains two equal values.

Heap sort is not stable. In Figure 13, we start with (34, A),(7, A),(7, B),(32, C),(7, C). When this array gets heap sorted by the first value, the end result is (7,C), (7, B), (7, A), (32, C), (34, A). The elements with key 7 show up in different order than they were in in the original array (A, B, C). This means that heap sort is unstable.

Problem 7: Comparing algorithms

Approach

I will use my already developed sorting algorithms to sort array A, sorted A, and reversed A, counting the number of comparisons they make. A main function will also measure the time in ms it takes to sort a random array of 1000, 5000, and 10000 integers (integers between 0-100000). It will print the first 10 elements in each sorted array to verify they are in correct order. A helper function will be used to run the sorting algorithms.

Pseudocode

A section with all of my previous sorting algorithms with added in comparison counts if they did not have it
    Bubble sort
    Insertion sort
    Selection sort
    Quick sort
    Merge sort
    Heap sort

```
# HELPER FOR RUNNING ALGOS
num_Algos = 6;
names[num_Algos] = "Bubble Sort", "Insertion Sort", "Selection Sort", "Quick Sort", "Merge Sort", "Heap Sort";

function runSort(algo, arr, int ):
    comparisons = 0
    other = 0
    if algo==0:
        bubbleSort(arr, n, comparisons, other)
    elif algo==1:
        insertionSort(arr, n, comparisons, other)
    elif algo==2:
        selectionSort(arr, n, comparisons, other)
    elif algo==3:
        quickSort(arr, 0, n-1, comparisons)
    elif algo==4:
        mergeSort(arr, 0, n-1, comparisons, other)
    else:
        heapSort(arr, n, comparisons, other)
    return comparisons

# MAIN
function main():
    # RECORD COMPARISONS
    define A, sorted A, revered A
    n = size(A)
    inputs = [A, sorted A, reversed A]
    print input[0], input[1], input[2]
    # set up table by printing Algorithm —- A — Sorted — Reversed
    for a from 0 to num_Algos:
        print names[a]
        for k from 0 to 3:
            copy = inputs[k]
            comps = runSort(a, copy, n)
            print comps

    # TIMING
    times= []
    results= []
    num_Sizes = 3
    int sizes[num_Sizes] = [1000, 5000, 10000]
    # set fixed seed
    # distribution of random numbers from 1-100000
    for s from 0 to num_Sizes:
        for i from 0 to sizes[s]:
            data[i] = distribution(randomnumbergen)
        for a from 0 to num_Algos:
```

```
        arr = data
        start a chrono timer
        runSort (a, arr.data(), sizes[s])
        end timer
        times[a][s] = measured time
        results[a][s] = arr
# make a table with columns Algorithm and sizes
print Algorithm on the left
for s from 0 to s ¡ num_Sizes:
    on the right print n=sizes[s]
for a from 0 to num_Algos:
    print names[a] on the left
    for s from 0 to num_Sizes:
        print times[a][s] on the right

# CHECK FIRST 10 ELEMENTS
for s from 0 to num_Sizes:
    for a from 0 to num_Algos:
        print names[a]
        printArray(results[a][s], 10)
```

Execution

```
(base) piperlentz@Pipers-MacBook-Pro-2 Assignment02_Lentz % ./Assigr
A:        34 7 23 32 5 62 14 19
Sorted:   5 7 14 19 23 32 34 62
Reversed: 62 34 32 23 19 14 7 5

Number of comparisons (n = 8)
Algorithm                A    Sorted  Reversed
Bubble Sort             25         7        28
Insertion Sort          20         7        28
Selection Sort          28        28        28
Quick Sort              14        28        28
Merge Sort              17        12        12
Heap Sort               26        27        24

Sort time in milliseconds (random integers 1-100000)
Algorithm            n = 1000    n = 5000   n = 10000
Bubble Sort             4.487      52.908     182.532
Insertion Sort          1.042      14.119      51.468
Selection Sort          1.306      25.181      99.652
Quick Sort              0.068       0.347       0.778
Merge Sort              0.562       2.510       5.437
Heap Sort               0.104       0.527       1.161

First 10 elements after sorting n = 1000:
Bubble Sort     : 280 290 293 294 701 743 860 979 1120 1150
Insertion Sort  : 280 290 293 294 701 743 860 979 1120 1150
Selection Sort  : 280 290 293 294 701 743 860 979 1120 1150
Quick Sort      : 280 290 293 294 701 743 860 979 1120 1150
Merge Sort      : 280 290 293 294 701 743 860 979 1120 1150
Heap Sort       : 280 290 293 294 701 743 860 979 1120 1150

First 10 elements after sorting n = 5000:
Bubble Sort     : 5 8 18 22 44 74 92 142 171 288
Insertion Sort  : 5 8 18 22 44 74 92 142 171 288
Selection Sort  : 5 8 18 22 44 74 92 142 171 288
Quick Sort      : 5 8 18 22 44 74 92 142 171 288
Merge Sort      : 5 8 18 22 44 74 92 142 171 288
Heap Sort       : 5 8 18 22 44 74 92 142 171 288

First 10 elements after sorting n = 10000:
Bubble Sort     : 14 17 20 35 45 51 61 63 92 99
Insertion Sort  : 14 17 20 35 45 51 61 63 92 99
Selection Sort  : 14 17 20 35 45 51 61 63 92 99
Quick Sort      : 14 17 20 35 45 51 61 63 92 99
Merge Sort      : 14 17 20 35 45 51 61 63 92 99
Heap Sort       : 14 17 20 35 45 51 61 63 92 99
```

Figure 15: Comparing copmarisons and times for each sorting algorithm.

Which algorithms grow like n^2 and which grow like n log n? Do your measurements agree with the theory? Which algorithm would you choose for a large random array, and which for an array that is already almost sorted? Explain your choices.

Bubble sort, insertion sort, and selection sort grow like n^2 because of nested for and while loops. Merge and heap sort grow like $nlogn$. Quick sort typically will sort on time $nlogn$, but in its worst case (an already sorted array) it will grow like n^2.

My measurements agree with the big O theory. We see in Figure 15 that bubble, insertion, and selection sort took about 4.5, 1.0, and 1.3 ms respectively to sort the random array of size 1000. Meanwhile, quick, merge, and heap sort took about 0.07, 0.6, and 0.1 ms respectively - all faster than the other 3 methods, like we expected.

I would choose quick sort for a large random array because it uses a low amount of comparisons (in our random array of size 8 it used 14 comparisons) and takes a short amount of time to sort (0.778 ms for a random array of size 10000). However, for an array that is already almost sorted, I would choose insertion sort because in its best case scenario (sorted/about sorted array), it only takes O(n) time. However, I would be cautious because if the out-of-order elements are far from their correct placements, it may take longer, approaching O(n^2). Merge sort is a

safer option, because in its best and worst case scenarios, it will take O($nlogn$) time. Also, we see in Figure 15 that in the sorted array, merge sort used 12 comparisons, much less than heap sort's 27 comparisons, which is why I'm choosing merge sort over heap sort.